CNS A.Y. 2026–27: Group 1 Technical Submission Report

Lightweight IoT Botnet Attack Detection Using Cost-Complexity Pruned CART Decision Trees

Subject: Cryptography and Network Security (CNS) | **Faculty:** Prof. Renuka Pawar
Institution: Sardar Patel Institute of Technology, Mumbai
Group: Group 1 — Decision Tree
Assigned Research Topic: Lightweight Android and IoT Botnet Detection Using C4.5 Decision Trees
Actual Implementation: CART Decision Tree (`sklearn.tree.DecisionTreeClassifier`)
Regularization: Minimal Cost-Complexity Pruning ($ccp_\alpha = 1.999753 \times 10^{-5}$)

Abstract

Internet of Things (IoT) devices deployed in residential and industrial environments frequently lack defensive software, rendering them prime targets for autonomous botnet infection by Mirai and BASHLITE malware. This investigation evaluates an optimized, cost-complexity pruned CART decision tree for multi-vector IoT botnet detection on the benchmark N-BaIoT dataset (115 continuous features, 11 traffic categories). Evaluated across 25,674 held-out test packets, the final pruned model achieves a multiclass accuracy of **99.6183%**, a macro F1-score of **99.6510%**, and a binary botnet detection accuracy of **99.9649%** (F1-score: **99.9810%**). By tuning ccp_α to 1.999753×10^{-5} via an internal validation partition, tree depth was reduced from 21 to 18 levels (-14.3%) and total node count was reduced from 181 to 133 (-26.5%), shrinking the train-test generalization gap from 0.1022% to 0.0769% (-24.8% overfitting). The model serialized footprint is **5.44 KB**, with a single-packet latency of **34.41 μs** and a batch processing throughput of **12,036,209 packets/sec**, demonstrating line-rate suitability for embedded edge gateways.

---

1. Problem Statement & Motivation

IoT appliances (e.g., surveillance cameras, smart doorbells, thermostats) operate under strict hardware budgets (128–512 MB RAM, sub-1 GHz CPUs). Deep neural networks and ensemble methods impose significant memory footprints and floating-point multiply-accumulate overheads that exhaust edge hardware. A decision tree provides an optimal alternative by mapping packet features through a bounded hierarchy of scalar comparisons, enabling microsecond execution and white-box auditability.

---

2. Dataset Description & Preprocessing

The N-BaIoT benchmark comprises network telemetry collected from 9 physical commercial IoT devices infected with Mirai and BASHLITE (Gafgyt) malware across 89 raw CSV files (7,062,606 packets).

- **Deterministic Systematic Sampling:** Applying a fixed step of $k = 55$ yielded **128,366 packets**, preserving class and device representation.
- **Traffic Taxonomy (11 Classes):** 1 benign baseline, 5 BASHLITE vectors (`combo`, `junk`, `scan`, `tcp`, `udp`), and 5 Mirai vectors (`ack`, `scan`, `syn`, `udp`, `udpplain`).

- **Feature Space:** 115 continuous statistics extracted via damped running windows across 5 decay factors ($\lambda \in {0.01, 0.1, 1, 3, 5}$) and 4 traffic scopes: Host (H), Host-to-Host (HH), Jitter (HH_jit), and Sockets (HpHp).
- **Transport Port Dynamics:** Discrete port numbers are absent; transport interactions are captured via continuous `HpHp_*` socket aggregates.
- **Hygiene:** 0 missing values, 0 infinite values.

---

3. Algorithmic Background & CART vs. C4.5

While the assigned topic specifies C4.5, our actual software implementation uses Python's `sklearn.tree.DecisionTreeClassifier`, which implements an optimized version of the Classification and Regression Trees (CART) algorithm: - **Splitting Criterion:** CART evaluates Gini Impurity ($\text{Gini}(t) = 1 - \sum p_i^2$), avoiding logarithmic transformations. - **Tree Topology:** Strictly binary recursive splitting ($x_j \le \theta$). - **Regularization:** Minimal Cost-Complexity Pruning ($R_\alpha(T) = R(T) + \alpha |T|$).

---

4. Cost-Complexity Pruning & Optimization Methodology

Unpruned trees overfit flow fluctuations. Breiman's weakest-link pruning defines the effective threshold for collapsing subtree T_t into leaf t: $$\alpha_{\text{eff}}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

To eliminate test leakage, a strict 3-way partitioning was enforced: - **Training Set (102,692 packets):** Internally split into 82,153 internal training and 20,539 internal validation samples. - **Held-Out Test Set (25,674 packets):** Sealed until final verification.

The pruning path discovered 60 candidate alphas. Evaluating 20 representative candidates against the validation set identified **Candidate 2 with $\alpha^* = 1.999753 \times 10^{-5}$**, which achieved peak validation Macro F1 (99.7696%) while pruning 9 redundant leaves.

---

5. Final Results & Comparative Evaluation

Baseline vs. Final Pruned Model Comparison

Evaluation Metric	Baseline (Unpruned, $\alpha=0.0$)	Final Pruned Model ($\alpha=1.9998e-5$)	Engineering Impact
Tree Depth	21 levels	**18 levels**	**-3 levels (-14.3%)**
Total Node Count	181 nodes	**133 nodes**	**-48 nodes (-26.5%)**
**Terminal Leaves ($	T	$)**	91 leaves
Multiclass Test Accuracy	99.6144%	**99.6183%**	**+0.0039% (Maintained)**
Multiclass Macro F1	99.6487%	**99.6510%**	**+0.0023% (Maintained)**

Evaluation Metric	Baseline (Unpruned, $\alpha=0.0$)	Final Pruned Model ($\alpha=1.9998e-5$)	Engineering Impact
Binary Detection Accuracy	99.9649%	**99.9649%**	**Identical (>99.96%)**
Binary Detection F1	99.9810%	**99.9810%**	**Identical (>99.98%)**
Generalization Gap	0.1022%	**0.0769%**	**-24.8% Overfitting Reduction**
Serialized Model Size	6.16 KB	**5.44 KB**	**-11.7% Storage Reduction**

Standardized Tournament Metrics

Tournament Metric	Measured Value	Standardized Evaluation Format
Accuracy	**99.6183%**	Percentage of Correct Classifications
Precision	**99.6611%**	Macro-Averaged Precision
Recall	**99.6455%**	Macro-Averaged Recall (Sensitivity)
F1-Score	**99.6510%**	Macro-Averaged Harmonic Mean
ROC-AUC	**0.999410**	One-vs-Rest Multiclass Macro ROC-AUC
Training Time	**5.8563 s**	Full Train Partition Single-Core Fitting
Inference Latency	**0.000187 ms/sample**	Vectorized Batch Latency (0.187 μs/pkt)

---

6. Confusion Matrix & Decision Path Analysis

Across 25,674 test packets, Mirai attack classes achieved near-perfect classification (>99.9% precision/recall). Out of 23,653 malicious packets, **23,650 were detected correctly** (3 misses). Out of 2,021 benign packets, **2,015 were classified correctly** (6 false alarms).

Top Gini features: 1. `H_L0.01_mean` (0.1811) 2. `MI_dir_L0.01_mean` (0.1724) 3. `HH_jit_L1_mean` (0.1701) 4. `MI_dir_L0.01_weight` (0.1365) 5. `MI_dir_L0.1_weight` (0.0865)

At Node 107, socket-level feature `HpHp_L1_weight` (importance 0.0568) successfully discriminates between Mirai UDP floods and Mirai UDP-plain attacks.

---

7. Computational Performance & Hardware Viability

- **Single-Sample Real-Time Latency: 34.41 μs**, easily supporting inline packet processing.
- **Batch Throughput: 12,036,209 packets/sec**, exceeding Gigabit Ethernet line rate (1.488M pkts/s) by 8x.
- **Memory Footprint:** 5.44 KB serialized binary, $< 100\text{ KB}$ active RAM.

---

8. Limitations & Threats to Validity

1. Subsampling to 128,366 rows was necessitated by local RAM limits.
2. Latency benchmarks evaluate tree walk; upstream Kitsune feature extraction requires separate processing.
3. Testbed telemetry may have lower background noise than active enterprise networks.

---

9. Conclusion

Cost-complexity pruned CART decision trees provide a mathematically grounded, highly accurate (99.62% multiclass, 99.96% binary), and computationally lightweight solution for IoT edge intrusion detection. Tuning ccp_α successfully reduced tree depth by 14.3% and eliminated 26.5% of nodes while mitigating overfitting.

---

10. References

1. Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, 2018.
2. M. Shafiq et al., "CORRAUC: A malicious bot-IoT traffic detection method in IoT network," *IEEE IoTJ*, 2021.
3. A. Moubayed et al., "Optimized machine learning-based network intrusion detection for IoT networks," *IEEE IoTJ*, 2020.
4. L. Breiman et al., *Classification and Regression Trees*, CRC Press, 1984.